

PostgreSQL Aplicado a la Enseñanza Técnica

■ Bootcamp: Transformación Digital para la Docencia Técnica

Guía Práctica para Participantes · Sesión Presencial · 4 horas

Prerrequisito: Conocimiento básico de bases de datos relacionales y PostgreSQL

#	Bloque	Tema	Duración
1	Bloque 1	Gestión de Usuarios en PostgreSQL	1h 15min
—		Receso	15 min
2	Bloque 2	Optimización de Consultas con IA	1h 15min
—		Receso	15 min
3	Bloque 3	Copias de Seguridad y Protección de Datos	1h

■■ Bases de datos que usaremos en esta sesión

test_school	Bloque 1 — Roles y permisos. Tablas: estudiantes, cursos
escuela_optimizacion	Bloque 2 — Optimización con EXPLAIN ANALYZE e índices

■ Gestión de Usuarios en PostgreSQL

Concepto Clave: Roles en PostgreSQL

Los roles existen a nivel de **servidor**, no de base de datos. Se crean una vez y luego se les asignan permisos capa por capa. Un rol puede tener LOGIN (usuario real) o no tenerlo (plantilla de permisos).

Nivel	Permiso que se otorga
Base de datos	CONNECT
Schema	USAGE
Tabla	SELECT, INSERT, UPDATE, DELETE

Paso 1 — Preparar la base de datos

Conéctate a tu servidor PostgreSQL y ejecuta lo siguiente en pgAdmin (Query Tool):

```
-- Crear la base de datos de práctica
CREATE DATABASE test_school;

-- Conectarte a test_school y crear la tabla
CREATE TABLE estudiantes (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100),
  correo VARCHAR(100)
);

INSERT INTO estudiantes (nombre, correo) VALUES
('Ana García', 'ana@escuela.edu'),
('Carlos López', 'carlos@escuela.edu'),
('María Torres', 'maria@escuela.edu');
```

Paso 2 — Crear rol de solo lectura y usuario

■ Un rol sin LOGIN es solo una "plantilla" de permisos. El usuario con LOGIN hereda esos permisos.

```
-- Rol contenedor (sin login, es una plantilla)
CREATE ROLE rol_lectura;

-- Capa 1: permiso de conexión a la BD
GRANT CONNECT ON DATABASE test_school TO rol_lectura;
-- Capa 2: permiso de ver el schema
GRANT USAGE ON SCHEMA public TO rol_lectura;
-- Capa 3: permiso de leer todas las tablas
GRANT SELECT ON ALL TABLES IN SCHEMA public TO rol_lectura;

-- Crear usuario real con LOGIN
CREATE ROLE docente01 WITH LOGIN PASSWORD 'Segura123!';
-- Asignarle el rol
GRANT rol_lectura TO docente01;
```

Paso 3 — Verificar los roles y permisos

```
-- Ver roles y si tienen login
SELECT rolname, rolcanlogin, rolsuper FROM pg_roles
WHERE rolname IN ('rol_lectura', 'docente01');

-- Ver que docente01 tiene asignado rol_lectura
SELECT r.rolname AS usuario, m.rolname AS rol_asignado
FROM pg_auth_members am
JOIN pg_roles r ON r.oid = am.member
JOIN pg_roles m ON m.oid = am.roleid
WHERE r.rolname = 'docente01';

-- Ver permisos sobre la tabla estudiantes
SELECT grantee, privilege_type
FROM information_schema.role_table_grants
WHERE table_name = 'estudiantes';
```

Paso 4 — Probar conexión como docente01

En pgAdmin, agrega un nuevo servidor con estas credenciales:

Campo	Valor
Host	localhost
Port	5432
Maintenance database	test_school
Username	docente01
Password	Segura123!

Desde esa conexión, prueba estas consultas:

```
-- ■ DEBE funcionar (tiene SELECT)
SELECT * FROM estudiantes;

-- ■ NO debe funcionar (no tiene INSERT)
INSERT INTO estudiantes (nombre, correo) VALUES ('Prueba', 'prueba@test.com');

-- ■ NO debe funcionar (no tiene DELETE)
DELETE FROM estudiantes WHERE id = 1;
```

Paso 5 — Demostración: Revocar y restaurar acceso

■ Haz esto desde tu conexión principal (postgres). El efecto se ve inmediatamente en la conexión de docente01.

```
-- Desde tu servidor (postgres): revocar acceso
REVOKE rol_lectura FROM docente01;

-- Desde docente01: ahora esto falla
SELECT * FROM estudiantes; -- ERROR: permission denied

-- Desde tu servidor (postgres): restaurar acceso
GRANT rol_lectura TO docente01;

-- Desde docente01: vuelve a funcionar
SELECT * FROM estudiantes; -- OK
```

■ ■ Ejercicio Práctico — Tres Perfiles de Acceso

Escenario: Una institución educativa necesita tres niveles de acceso distintos a su base de datos.

Perfil	Usuario	Qué necesita
admin_bd	admin_bd	Acceso total a test_school
Docente	docente02	Solo SELECT en estudiantes y cursos
Estudiante	ana	Solo ver su propio registro (RLS)

Perfil 1: admin_bd — Acceso total

```
CREATE ROLE admin_bd WITH LOGIN PASSWORD 'Admin123!';
GRANT ALL PRIVILEGES ON DATABASE test_school TO admin_bd;
GRANT ALL PRIVILEGES ON SCHEMA public TO admin_bd;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_bd;

-- Verificar
SELECT rolname, rolcanlogin, rolsuper FROM pg_roles WHERE rolname = 'admin_bd';
```

Perfil 2: docente02 — Solo lectura en estudiantes y cursos

```
-- Primero crear la tabla cursos
CREATE TABLE cursos (id SERIAL PRIMARY KEY, nombre VARCHAR(100), codigo VARCHAR(20));
INSERT INTO cursos (nombre, codigo) VALUES ('Matemáticas', 'MAT-01'), ('Programación', 'PRG-01');

-- Crear rol y usuario docente
CREATE ROLE rol_docente;
GRANT CONNECT ON DATABASE test_school TO rol_docente;
GRANT USAGE ON SCHEMA public TO rol_docente;
GRANT SELECT ON estudiantes, cursos TO rol_docente;

CREATE ROLE docente02 WITH LOGIN PASSWORD 'Docente123!';
GRANT rol_docente TO docente02;
```

Perfil 3: estudiante (ana) — Solo su propio registro con Row Level Security

■ RLS = Row Level Security. PostgreSQL agrega un WHERE invisible en cada consulta según el usuario conectado.

```
-- Agregar columna de usuario a la tabla
ALTER TABLE estudiantes ADD COLUMN usuario VARCHAR(100);
UPDATE estudiantes SET usuario = 'ana' WHERE nombre = 'Ana García';
UPDATE estudiantes SET usuario = 'carlos' WHERE nombre = 'Carlos López';
UPDATE estudiantes SET usuario = 'maria' WHERE nombre = 'María Torres';

-- Activar RLS en la tabla
ALTER TABLE estudiantes ENABLE ROW LEVEL SECURITY;

-- Crear la política: cada usuario solo ve su fila
CREATE POLICY ver_propio_registro ON estudiantes
FOR SELECT USING (usuario = current_user);

-- Crear usuario ana
CREATE ROLE ana WITH LOGIN PASSWORD 'Ana123!';
GRANT CONNECT ON DATABASE test_school TO ana;
GRANT USAGE ON SCHEMA public TO ana;
GRANT SELECT ON estudiantes TO ana;

-- Conectarse como ana y probar
SELECT * FROM estudiantes; -- Solo muestra la fila de Ana
```

■■ El superusuario postgres siempre ve todas las filas aunque RLS esté activo. RLS solo aplica a usuarios regulares.

■ Optimización de Consultas con IA

Concepto Clave: EXPLAIN ANALYZE

EXPLAIN muestra el plan de ejecución sin ejecutar la consulta. **EXPLAIN ANALYZE** lo ejecuta y muestra los tiempos reales.

El objetivo es detectar **Seq Scan** en tablas grandes y reemplazarlos con **Index Scan**.

Paso 1 — Preparar base de datos con 10,000 registros

Conéctate a `escuela_optimizacion` y ejecuta:

```
CREATE DATABASE escuela_optimizacion;

-- Conectarte a escuela_optimizacion y crear tabla
CREATE TABLE estudiantes (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100),
  correo VARCHAR(100),
  carnet VARCHAR(20),
  fecha_ingreso DATE
);

-- Insertar 10,000 registros de prueba
INSERT INTO estudiantes (nombre, correo, carnet, fecha_ingreso)
SELECT
  'Estudiante ' || i,
  'est' || i || '@escuela.edu',
  'C-' || LPAD(i::text, 5, '0'),
  CURRENT_DATE - (random() * 1000)::int
FROM generate_series(1, 10000) AS i;
```

Paso 2 — Consulta SIN índice — ver el problema

```
EXPLAIN ANALYZE
SELECT * FROM estudiantes WHERE correo = 'est500@escuela.edu';
```

Qué buscar en el resultado	Qué significa
Seq Scan on estudiantes	■■ Leyó TODA la tabla (10,000 filas)
Rows Removed by Filter: 9999	■■ Descartó 9,999 filas para encontrar 1
Execution Time: ~1.7 ms	Parece rápido, pero con millones de filas es crítico

Paso 3 — Crear índice y comparar

```
-- Crear índice sobre la columna correo
CREATE INDEX idx_estudiantes_correo ON estudiantes(correo);

-- Repetir exactamente la misma consulta
EXPLAIN ANALYZE
SELECT * FROM estudiantes WHERE correo = 'est500@escuela.edu';
```

	Sin índice	Con índice
Método	Seq Scan	Index Scan
Filas descartadas	9,999	0
Execution Time	~1.732 ms	~0.142 ms
Velocidad	—	12x más rápido

■ Usando IA para Analizar el Plan de Ejecución

La IA puede ayudarte a interpretar el resultado de EXPLAIN ANALYZE. El flujo recomendado es: ejecutás la consulta → copiás el resultado → se lo pedís a la IA.

Prompt listo para usar:

```
Soy docente de PostgreSQL nivel básico.
Tengo esta consulta:

SELECT * FROM estudiantes WHERE correo = 'est500@escuela.edu';

Y este es el resultado de EXPLAIN ANALYZE:
[pegá el resultado aquí]

¿Qué problemas ves y cómo puedo mejorar el rendimiento?
Explicame en términos sencillos.
```

■■ La IA puede sugerir cosas útiles pero también incorrectas. Siempre probá cualquier sugerencia en un entorno de desarrollo antes de aplicar en producción.

Paso	Acción
1	Copió la consulta lenta al chat de IA
2	Pegó el resultado de EXPLAIN ANALYZE
3	Pedí análisis y sugerencias en términos simples
4	Evalué la sugerencia — ¿tiene sentido para tus datos?
5	Probé en desarrollo antes de aplicar en producción

■ Copias de Seguridad y Protección de Datos

Concepto Clave: La Regla 3-2-1

3 copias de los datos

en 2 medios distintos

1 copia fuera del sitio

■■ Configurar Terminal en Windows

Los comandos de backup se ejecutan en CMD, no en pgAdmin. Primero verifiqué que funcione:

```
-- En CMD de Windows, verificar versión
pg_dump --version
```

■■ Si el comando no se reconoce, agregar al PATH: Variables de entorno → Variables del sistema → Path → Nuevo → C:\Program Files\PostgreSQL\18\bin (ajustar según tu versión). Cerrar y reabrir CMD.

Opciones de Backup

Opción	Comando	Para qué sirve
1 — SQL legible	<code>pg_dump -f archivo.sql</code>	Backup completo, se puede abrir con notepad
2 — Comprimido	<code>pg_dump -Fc -f archivo.dump</code>	Recomendado para producción, pesa menos
3 — Solo estructura	<code>pg_dump --schema-only</code>	Sin datos, útil para documentación
4 — Servidor completo	<code>pg_dumpall</code>	Incluye usuarios, roles y todas las BDs

Paso 1 — Ejecutar los backups

■ Ejecutá estos comandos en CMD. El archivo se guarda en la carpeta donde estés ubicado. Te pedirá la contraseña de postgres.

```
-- Opción 1: Backup SQL legible
pg_dump -U postgres -d test_school -f backup_testschool.sql

-- Opción 2: Backup comprimido (recomendado producción)
pg_dump -U postgres -d test_school -Fc -f backup_school.dump

-- Opción 3: Solo estructura
pg_dump -U postgres -d test_school --schema-only -f estructura_school.sql

-- Opción 4: Todo el servidor (incluye usuarios y roles)
pg_dumpall -U postgres -f backup_total_completo.sql

-- Ver los archivos creados y su tamaño
dir backup_*
```

Paso 2 — Restaurar un backup

■ ■ Siempre restaura en una BD de prueba primero. Nunca sobre la BD de producción directamente.

```
-- Crear BD de prueba para restaurar
psql -U postgres -c "CREATE DATABASE escuela_restaurada;"

-- Restaurar desde .sql
psql -U postgres -d escuela_restaurada -f backup_testschool.sql

-- Restaurar desde .dump (formato comprimido)
pg_restore -U postgres -d escuela_restaurada backup_school.dump

-- Verificar en pgAdmin que la tabla existe con datos
```

■ Checklist de Buenas Prácticas

Revisión grupal — ¿Tu institución cumple con esto?

■	Se realizan backups de forma automática y periódica
■	Los backups se almacenan en una ubicación distinta al servidor
■	Se prueba la restauración del backup al menos una vez al mes
■	Los backups están protegidos con contraseña o cifrado
■	Solo personal autorizado tiene acceso a los archivos de backup
■	Existe documentación del proceso de recuperación ante desastres
■	Las contraseñas de la BD no están en texto plano en los scripts
■	Se registran (logs) los accesos a la base de datos

■ Repaso Final — Lo más importante que aprendí hoy

Tema	Lo más importante que aprendí hoy
Gestión de usuarios	
Optimización con IA	
Copias de seguridad	